$ curl /users/123 | usecase | repository | db

Clean Architecture

// □□□□□□□□□

Controller	Usecase	Repository	Presenter
□務生	主廚	倉管	打包員

```
$ cat blueprint.md
```

□日路線

01 / [illegible]

□黏在一起的程式怎麼讓你崩潰

02 / [illegible]

服務生、主廚、倉管、打包員

03 / [illegible]

TypeScript [illegible] + [illegible]

04 / [illegible]

[illegible] DB[illegible]——[illegible]

核心規則只有一條：你只跟上一層說話，下層的細節不關你的事。

PART 01

□什麼需要它?

□□□□□□□□□□□□

全黏在一起

```javascript
// UI 直接連資料庫
app.get('/users', (req, res) => {
  const users = db.query('SELECT * FROM users');
  res.json(users.map(u => ({
    name: u.name,
    email: u.email,
    createdAt: u.created_at.format('YYYY-MM-DD')
  })));
});
```

INCOMING REQUESTS

老闆：「我們要換 GraphQL！」

→ 你崩潰（路由、格式全重寫）

老闆：「加個快取！」

→ 改 10 個地方

老闆：「換 MongoDB！」

→ SQL 散落各處，找都找不完

分層之後：改「怎麼做」，不動「做什麼」

資料庫?
Repository

API 格式?
Presenter

前端框架?
Controller

INVARIANT

商業邏輯（Usecase）永遠不動 —— 它是你的核心資產，活得比任何框架久。

PART 02

四個角色

餐廳編制

Controller = 服務生

接單、把參數轉成程式能用的格式、傳話給廚房。不能寫商業邏輯!

Usecase = 主廚

[illegible]——[illegible] JSON [illegible] XML

Repository = 倉管

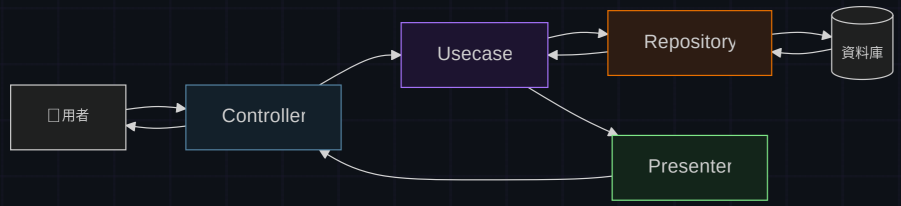
[illegible] MySQL、MongoDB [illegible] API 拿？主廚不關心

Presenter = 打包員

JSON [illegible] XML 盒子？只管怎麼呈現，不管資料哪來

重點：廚房不知道菜市場在哪。大家只管自己的事。

請求流程：資料只往內流



去程

UI → Usecase → Repository → DB

回程

DB → Repository → Usecase → Presenter → UI 永遠不反向

PART 03

TypeScript □□

□□□□□□□□□□□□

Repository 先定契約

```ts
export interface UserRepository {
  getUser(id: number): Promise<User | null>;
}

// 實作: 用 MySQL —— Usecase 完全不知道
export class MySQLUserRepository
  implements UserRepository {
  async getUser(id: number) {
    const rows = await db.query(
      'SELECT * FROM users WHERE id = ?', [id]);
    return rows[0] ?? null;
  }
}
```

Usecase 商業邏輯的家

```ts
export class GetUserUseCase {
  constructor(
    private userRepository: UserRepository
  ) {}

  async execute(id: number) {
    const user =
      await this.userRepository.getUser(id);
    if (!user) return null;

    // 真正的商業邏輯: 30 天未登入視為無效
    if (user.createdAt < thirtyDaysAgo()) {
      return null;
    }
    return user;
  }
}
```

依賴介面，不依賴實作

[illegible] UserRepository —— MySQL [illegible] Mongo [illegible] 不知道也不需要知道

Presenter ∥ Controller

```ts
// Presenter只管怎麼呈現
export class UserPresenter {
  static toResponse(user: User | null) {
    if (!user) return { error: 'User not found' };
    return {
      id: user.id,
      name: user.name,
      created_at: user.createdAt.toISOString(),
    };
  }
}
```

組裝端：依賴注入

```ts
// routes/user.ts —— 具體選擇集中在這
const repo = new MySQLUserRepository();
const usecase = new GetUserUseCase(repo);
const controller = new UserController(usecase);

router.get('/users/:id', controller.getUser);
```

COMPOSITION ROOT

Controller 不知道資料庫是什麼、Usecase 不知道前端要什麼 —— 每層都只跟介面說話。

PART 04

驗收時間

□□□□□□□□□□□□□□□

三個需求，三次「只改一層」

□□□ MongoDB□」

□□ MongoDBUserRepository □□□□□□□□□□□ new。其他層零改動。

□□□□ XML□」

□□ Presenter □ toResponse()。其他層零改動。

「查詢太慢，加快取！」

CacheUserRepository □□ MySQLUserRepository□□□□□□□□□□□□□□ DB。Usecase 完□沒動。

$ 每個需□都只動一層 —— 這就是□層的投資報酬□。

CHECKPOINT_01

使用者抱怨查詢太慢，你決定加快取（Cache）。該改哪一層？

A Usecase：在商業邏輯裡加一段查快取的程式

B Controller：收到請求先看看快取有沒有

C Repository：包一層 CacheUserRepository，先查快取再查 DB

FAQ 快問快答

[illegible] MVC 嗎?

MVC 常把邏輯與存取黏在一起；Clean Architecture 是把「做什麼」和「怎麼做」分家。

寫這麼多檔案不累嗎?

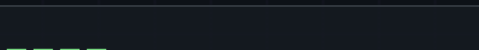
[illegible] 10 [illegible] 100 [illegible] 1 [illegible] 100 [illegible]

小工具也要這樣?

[illegible] 2 [illegible]

Repository 要寫介面嗎?

[illegible] Usecase [illegible]——[illegible]



□日重點

一條規則

資料流只能往內，不能往外

四個角色

□□□□□□□□□□□□□——□□□□

依賴介面

具體選擇集中在組裝端（依賴注入）

驗收標準

□ DB□□□□□□□□——□□□□□

□層，但不互相認識 —— 商業邏輯永遠不動。

Layers separated.

□□□□Spring Boot × DDD□□□□□□□□□□□□□□

```
$ curl /users/123
controller → usecase → repository → presenter
```